FIRST EXERCISE: SUM OF THE FIRST n NATURAL NUMBERS

Let's explore our first exercise: calculating the sum of the first n natural numbers. This classic problem introduces fundamental concepts in algorithmic thinking and serves as an excellent starting point for understanding recursion.

PROBLEM STATEMENT

Create a function that returns the sum **S(n) = 1 + 2 + 3 + ... + n** using ***recursion with simple if-else*** logic.

Example: S(5) = 1 + 2 + 3 + 4 + 5 = 15

MATHEMATICAL FOUNDATION

Before diving into code implementation, let's understand the mathematical foundation:

The sum of the first n natural numbers can be expressed as:

S(n) = 1 + 2 + 3 + ... + n

This can also be written as S(n) = n + (n-1) + (n-2) + ... + 2 + 1

There's also a well-known mathematical formula for this sum: $S(n) = n(n+1)/2$

However, for this exercise, we'll focus on the recursive definition:

Base case: S(0) = 0 (the sum of no numbers is 0)

Recursive case: S(n) = n + S(n-1)

RECURSION EXPLAINED (again)

Recursion is a technique where a function calls itself to solve smaller instances of the same problem. For this to work properly, we need:

1. **Base case(s)**: Condition(s) where the function returns a value without further recursive calls
2. **Recursive case(s)**: Where the function calls itself with a smaller problem

For our sum problem:

When n = 0, we return 0 (base case)

Otherwise, we add n to the sum of all numbers before it, S(n-1) (recursive case)

IMPLEMENTATION IN PYTHON

```python
def recursive_sum(n):
    # Base case: sum up to 0 is 0
    if n == 0:
        return 0
    else:
        # Recursive case: n + sum up to (n-1)
        return n + recursive_sum(n - 1)

print(recursive_sum(5))  # Output: 15
```

IMPLEMENTATION IN C

```c
#include <stdio.h>

int recursive_sum(int n) {
    if (n == 0)
        return 0;  // Base case
    else
        return n + recursive_sum(n - 1);  // Recursive call
}

int main() {
```

```
    printf("%d\n", recursive_sum(5));  // Output: 15
    return 0;
}
```

IMPLEMENTATION IN BASH

```
recursive_sum() {
    local n=$1
    if [ "$n" -eq 0 ]; then
        echo 0
    else
        # Recursive call and summation
        local prev_sum=$(recursive_sum $((n - 1)))
        echo $((n + prev_sum))
    fi
}

recursive_sum 5  # Output: 15
```

EXECUTION TRACE

Let's trace through the execution of recursive_sum(5) to understand how recursion works:

1. Call recursive_sum(5)

```
Since 5 ≠ 0, return 5 + recursive_sum(4)
But we need to compute recursive_sum(4) first
```

2. Call recursive_sum(4)

```
Since 4 ≠ 0, return 4 + recursive_sum(3)
But we need to compute recursive_sum(3) first
```

3. Call recursive_sum(3)

```
Since 3 ≠ 0, return 3 + recursive_sum(2)
But we need to compute recursive_sum(2) first
```

4. Call recursive_sum(2)

```
Since 2 ≠ 0, return 2 + recursive_sum(1)
But we need to compute recursive_sum(1) first
```

5. Call recursive_sum(1)

```
Since 1 ≠ 0, return 1 + recursive_sum(0)
But we need to compute recursive_sum(0) first
```

6. Call recursive_sum(0)

```
Since 0 = 0, return 0 (base case reached)
```

7. Now we can work our way back up:

```
recursive_sum(1) = 1 + 0 = 1
recursive_sum(2) = 2 + 1 = 3
recursive_sum(3) = 3 + 3 = 6
recursive_sum(4) = 4 + 6 = 10
recursive_sum(5) = 5 + 10 = 15
```

The final result is 15, which is indeed the sum of 1 + 2 + 3 + 4 + 5.

MEMORY VISUALISATION

When a recursive function executes, each call is stored in the programme's call stack:

```
Call Stack (grows downward):
+-------------------+
| recursive_sum(5)  |
+-------------------+
```

```
| recursive_sum(4)  |
+-------------------+
| recursive_sum(3)  |
+-------------------+
| recursive_sum(2)  |
+-------------------+
| recursive_sum(1)  |
+-------------------+
| recursive_sum(0)  |
+-------------------+
```

Once the base case is reached, the stack unwinds, calculating the results as it goes:

Results (calculated from bottom up):

```
recursive_sum(0) = 0
recursive_sum(1) = 1 + 0 = 1
recursive_sum(2) = 2 + 1 = 3
recursive_sum(3) = 3 + 3 = 6
recursive_sum(4) = 4 + 6 = 10
recursive_sum(5) = 5 + 10 = 15
```

COMPLEXITY ANALYSIS

For this recursive solution:

- **Time Complexity**: O(n) because each number from n down to 1 is processed exactly once
- **Space Complexity**: O(n) due to the recursion stack, which at its deepest will have n+1 frames

CONSIDERATIONS AND LIMITATIONS

While recursion provides an elegant solution, it has limitations:

1. **Stack Overflow**: For large values of n, the recursion depth might exceed the stack size limit, causing a stack overflow error.
2. **Performance**: Recursive solutions often perform slower than iterative ones due to function call overhead.
3. **Readability vs. Efficiency**: Though recursive code can be more readable and directly map to the mathematical definition, iterative solutions are typically more efficient.

ITERATIVE ALTERNATIVE

For comparison, here's an iterative solution:

```
def iterative_sum(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total
```

The iterative version avoids potential stack overflow and generally performs better for large inputs.

MATHEMATICAL SOLUTION

For completeness, the direct mathematical formula:

```
def mathematical_sum(n):
    return n * (n + 1) // 2
```

This is the most efficient solution with O(1) time and space complexity.

EDUCATIONAL VALUE

This exercise is valuable for teaching:

1. Basic recursion concepts
2. The relationship between mathematical formulas and algorithmic implementations

3. How to break down a problem into base and recursive cases
4. Understanding the call stack and execution flow in recursive functions

As students progress, they can compare this recursive approach with iterative and mathematical solutions, analysing the trade-offs in terms of code simplicity, performance, and memory usage.